UMHackathon 2026: Preliminary Round Deliverable

SYSTEM ANALYSIS DOCUMENTATION (SAD)

Domain: 2

Group Name: Palindrome

Group ID: 193

Project Name: SmartStock AI

Group Members

Name	Role	University
Maha Vaarshinie A/P Arupatha Rajoo	Team Leader	Multimedia University
Nareenthiran A/L Bhuvaneswaran	Team Member	Multimedia University
Yasha Durgashine	Team Member	Multimedia University

Table of Contents

1. Introduction....4

1.1. Purpose....4

1.2. Background.... 4

1.3. Target Stakeholder.... 5

2. System Architecture & Design....6

2.1. High Level Architecture.... 6

2.1.1 Overview:....6

2.1.2. LLM as Service Layer.... 6

2.1.3 Dependency diagram.... 7

2.2.2 Sequence Diagram.... 9

2.2 Technological Stack.... 11

2.2.1 Frontend.... 11

2.2.2 Backend....11

2.2.3 Database....12

2.2.4 Artificial Intelligence (Reasoning Engine).... 12

2.2.4 Orchestration & Networking....13

2.3 Key Data Flows....14

2.3.1....14

2.3.1.1 Data Flow Diagram (DFD).... 14

2.3.2. Normalized Database Schema.... 16

3. Functional Requirements & Scopes.... 18

3.1 Minimum Viable Product (MVP).... 18

3.2 Non-Functional Requirements (NFRs)....19

3.3 Future Improvements / Out of Scope....19

4. Monitor, Evaluation, Assumptions & Dependencies.... 21

4.1 Technical Evaluation.....21

4.1.1 Grayscale Rollout & A/B Testing.....21

4.1.2. Emergency Rollback & Golden Release.....21

4.1.3 Priority Matrix.....22

4.2 Monitoring.....22

4.3 Assumptions.....23

5. Project Management & Team Contributions.....24

5.1 Project Timeline (10 Days).....24

5.2 Team Members' Role.....24

5.3 Recommendations.....25

1. Introduction

This section presents the strategic decision intelligence solution, SmartStock AI, as the solution to the identified issues in the fragmented data and manual inventory management found in the Product Review Documentation.

1.1. Purpose

The system analysis report sheds light on the technical scope and design decisions that underlined the development of SmartStock AI.

The major aspects of the system discussed are as follows:

1. **Architecture:**
   - It is a cloud-native platform. The services are split into Microservices, running on AWS, and are independent of each other to provide high availability and modularity.
2. **Data Flows:**
   - Describes the flow of sales information and unstructured contextual notes flowing out of the dashboard of the SME owner, through the FastAPI backend, into the PostgreSQL in a persistent format, and out as optimized stock strategies.
3. **Model Process:**
   - The end-to-end workflow of core features is illustrated in the model: ingested multi-modal data, demand predictions formed through the ILMU GLM-5.1, explainable reasoning logs presented.
4. **Role of Reference:**
   - The following document will be used as a guide by all the developers and testers so they can know the high-level architecture and the required integration of the Z AI reasoning model.

1.2. Background

The current state of the Malaysian SME market is that manual or intuitive processes are being used in optimizing inventory, and this causes a massive waste of capital.

1. **Previous Version:**
    - Current solutions are usually fixed accounting software (e.g., old POS systems) that are run as a single application that does not have predictive intelligence.
2. **Changes in Major Architectural Components:**
    - Moving to an Agentic Mesh based on Microservices, rather than a monolithic, data-storage model only.
    - Addition of a special Reasoning Orchestrator to manage LLM API traffic.
3. **New Capabilities Introduced:**
    - Self-driving Demand Forecasting: Predicting stocks with the context-sensitive ILMU GLM-5.1.
    - Explainable Reasoning Logs: Making the chain of thought of any business recommendation made by AI visible.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
SME Owner	Uploads sales CSVs and text notes; reviews AI-driven "Pivot" suggestions.	Seamless stock optimization and clear, understandable decision explanations.
Development Team	Build and maintain the React.js and FastAPI platform.	Clear API usage contracts, modular codebase, and well-documented architecture.
QA Team	Validating system behavior across all user roles and edge cases.	Defined request/response formats and high test coverage for AI reasoning.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1 Overview:

Type	Details
System	Full-Stack Web Application
Architecture	Client-Server with RESTful API

The SmartStock AI platform is designed with a strict decoupling approach using the Microservices architectural style. For portability and consistency across the development environment during the ten-day sprint, the platform runs in isolation through Docker Compose:

- **Frontend Client:** A standalone React.js client running on port 5173, utilizing Vite to enable Hot Module Replacement.
- **Backend Reasoning Orchestrator:** A powerful FastAPI server hosted on port 8000, acting as the centralized endpoint for data processing and AI reasoning tasks.

These isolated microservices interact only through REST API calls. The decoupling strategy allows the intensive computation of sales CSV parsing and multistep Z.AI GLM-5.1 reasoning to be performed without affecting the user interface for the SME owner.

2.1.2. LLM as Service Layer

The SmartStock AI architecture incorporates the use of the ILMU GLM-5.1 as a core layer of service which serves as the main reasoning engine of demand prediction and decision intelligence.

2.1.3 Dependency diagram

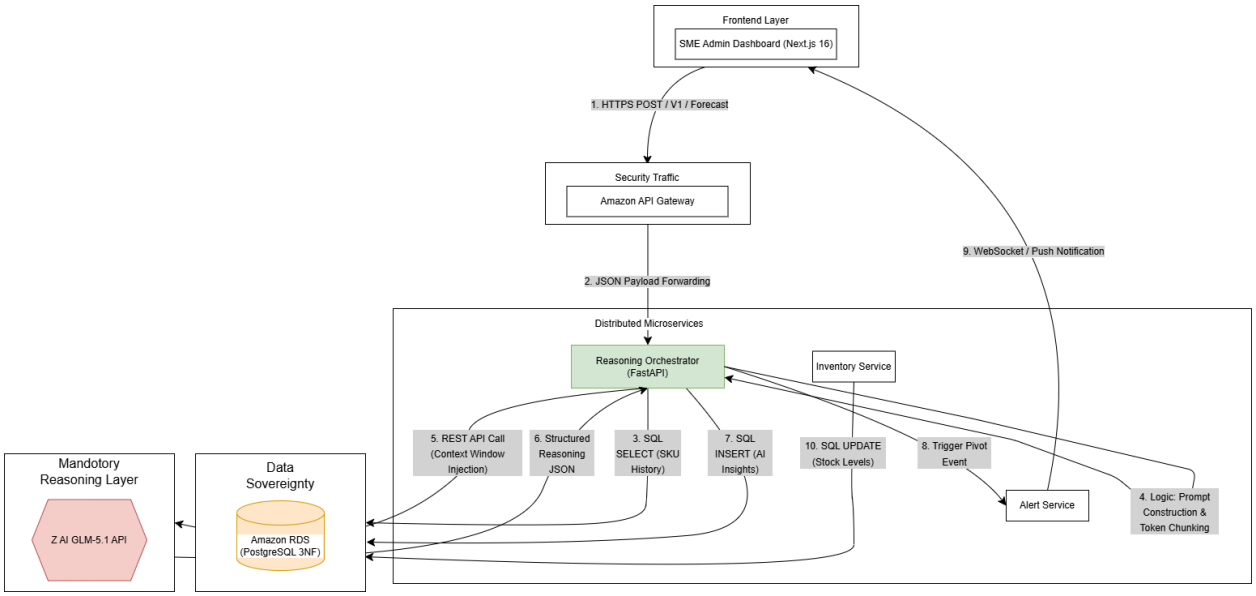

Figure 2: Dependency diagram

a. Prompt Construction & GLM Transmission

The central station of prompt engineering is the Reasoning Orchestrator (FastAPI). It dynamically combines user provided unstructured notes with historical information retrieved using SQL SELECT on Amazon RDS (Step 3). This joint payload is packaged into a structured prompt template and sent to the ILMU GLM-5.1 API through a secure REST API Call (Step 5).

b. Context Window Composition

Multimodal data points are added to the context window to guarantee high-accuracy demand forecasting. This includes:

- Structured Data: Database 3-month historical sales trends and SKU stock levels.
- Unstructured Context: SME owner comments on contextual factors, such as weather or supplier interruptions.

c. Receiving, Parsing, Passing of Response

The GLM sends a Structured Reasoning JSON (Step 6) to the system. The Reasoning Orchestrator then interprets this JSON to get certain actionable insights, including Restock Priority and Explanation of Logic. These insights are then:

- Inserted into the database using SQL INSERT (Step 7).
- Sent to the Alert Service (Step 8) to raise user alarms.

d. Data Integrity & Token Boundary Mitigation Script

For purposes of stability, cost management, and preventing any context window limit failure of the ILMU GLM-5.1, the FastAPI backend employs an intensive Token Boundary Mitigation Script. Prior to sending the Fused Context Window (Historical Data + User Notes) through the external REST API request, the programmatic Python backend calculates the number of tokens used according to the following formula:

$$\textit{Estimated Tokens} = \textit{Words X 1.3}$$

How it works:

- **Proactive Payload Filtering:** In the event that the number of estimated tokens surpasses the 1500 Token Context Window Limit, the script automatically catches the payload before it is sent.
- **Intelligent Chunk Truncation:** The script utilizes a "sliding window" principle, whereby it truncates any historical CSV data beyond the 7-day rolling period.
- **System Durability:** With this safeguard in place, the system will never encounter a 413 Request Entity Too Large error, nor model context overflow, and always receives response times of less than 5 seconds.

e. Significant API Calls between Components.

- The system employs the following main communication channels:
- Frontend Gateway: POST Https/v1/forecast starts the analysis process.
- Backend External: REST API Call gives the context to the ILMU GLM-5.1 API.

- Service Database: SQL SELECT/INSERT/UPDATE makes sure the data is persistent in Amazon RDS.
- WebSocket / Push Notification: Alert Frontend delivers real-time alerts to a user in the form of pivot notifications.

f. Component Interactions & Persistence

The Frontend Layer serves as the user interface, and it interacts with Amazon API Gateway to provide security and traffic control. The core logic is done through the Distributed Microservices (FastAPI), and the single source of truth is the Amazon RDS (PostgreSQL 3NF), which guarantees that all the stock updates (Step 10) and AI-generated strategies are stored in a persistence way.

g. Distributed Components & Deployment

SmartStock AI is intended to be deployed on AWS in a cloud-native manner. The system is split into autonomous microservices (Inventory, Reasoning and Alert) which are hosted as distributed parts. This is to make sure that the Reasoning Orchestrator would be able to scale on its own depending on the complexity of the AI inference tasks without impacting the core inventory tracking.

2.2.2 Sequence Diagram

a. Technical Walkthrough of the Interaction Flow

The sequence below shows the end-to-end functionality of the SmartStock AI core reasoning engine, including starting with initial data ingestion and ending with the ultimate strategy delivery.

Phase 1: Data Ingestion & Preparation

1. Request Initiation (Step 1-2): The SME Owner uploads structured sales data (csv) and unstructured contextual notes through the React.js Dashboard. This payload is safely passed across the Amazon API Gateway to the Reasoning Orchestrator.

2. Contextual Enrichment (Step 3-4): To curb AI hallucinations and to base the logic on fact, the orchestrator runs a SQL SELECT query to Amazon RDS to obtain the past 3 months of SKU-specific historical data.

Phase 2: Agentic Reasoning (Mandatory Layer)

3. Prompt & Token Logic (Step 5): The system carries out some internal processing to create a high density prompt. At this point, token restrictions are imposed and the inputs are divided into chunks that fit the requirements of the ILMU GLM-5.1 context window.
4. Model Inference (Step 6-7): The orchestrator triggers a call to the required ILMU GLM-5.1 API via a REST API. The model reasons in multiple steps using the fused data (historical sales + weather/event notes) and provides a Structured Reasoning JSON.

Phase 3: Persistence & Alerting

5. Data Sovereignty (Step 8): Data persistence Before the system delivers data to users, a SQL INSERT is executed to insert the AI-generated insights and predictions into the PostgreSQL 3NF database.
6. Real-Time Delivery (Step 9): The Alert Service sends a WebSocket push notification which transports the updated dashboard of the SME Owner at once with the new strategy of the Pivot and the log of explainable reasoning.

b. Sequence Diagram

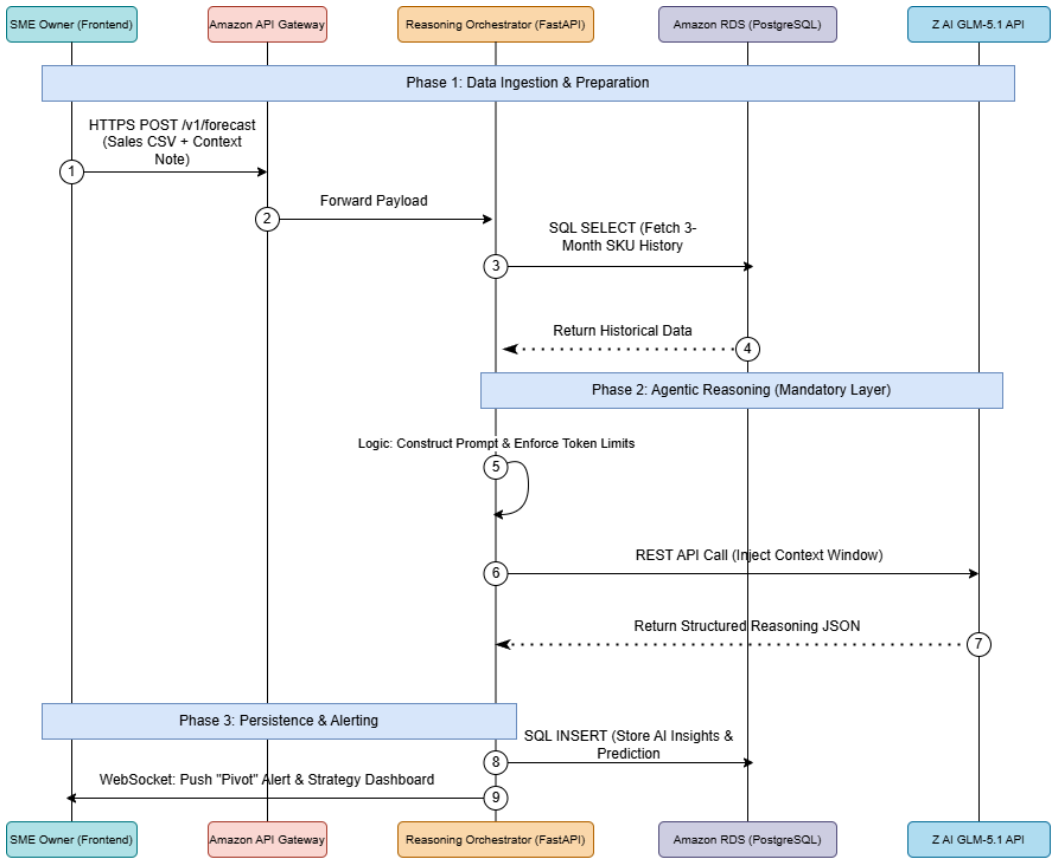

Figure 3: Sequence Diagram

2.2 Technological Stack

2.2.1 Frontend

Component	Selection	Justification
Framework	React.js (Vite)	Initialized via Vite for highly optimized build speeds and Hot Module Replacement (HMR) during the 10-day sprint.

Styling	Vanilla CSS	Utilizing CSS Variables and Glassmorphism for a premium, dark-mode enterprise dashboard without the overhead of heavy UI kits.
HTTP Client	Axios	Handles asynchronous REST API requests with optimized interceptors for tracking AI inference states.
Data Visualization	Recharts	Specifically integrated to power the 7-day predictive dashboard, rendering dynamic Area and Bar charts to visualize AI-predicted demand versus historical sales.

2.2.2 Backend

Component	Selection	Justification
Core Framework	Python, FastAPI	High-performance ASGI framework; chosen for its native support for asynchronous microservices and automated data validation.
Web Server	Uvicorn	A lightning-fast ASGI server implementation that ensures high concurrency for API requests.
Data Ingestion	Python-Multipart	Engineered to parse user-uploaded sales CSV files directly in-memory (via csv and io) to prevent server blocking.

2.2.3 Database

Component	Selection	Justification
Database Engine	SQLite	A lightweight, serverless relational database used to eliminate infrastructure friction during the rapid development phase.
ORM Layer	SQLAlchemy	Abstracts all database interactions, allowing for a seamless transition to AWS RDS (PostgreSQL) by simply updating the environment URL.

2.2.4 Artificial Intelligence (Reasoning Engine)

Component	Selection	Justification
Core Model	ILMU GLM-5.1	The central engine for all decision-intelligence tasks.
Integration Role	Reasoning Service	Fuses structured database arrays with unstructured user notes to output a strategic decision matrix and trade-off analysis.

2.2.4 Orchestration & Networking

Component	Selection	Justification

Containerization	Docker Compose	Orchestrates the decoupled Microservices architecture. It creates strictly isolated containers for the frontend and backend environments, ensuring parity between development and production.
Networking & Security	Cloudflare Zero Trust Tunnels	Securely exposes the locally hosted backend API to the public internet without opening inbound firewall ports, ensuring safe communication between the decoupled frontend client and the backend reasoning engine.

2.3 Key Data Flows

2.3.1

2.3.1.1 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) shows the overall process of transforming raw SME business data into decision-making intelligence. It is a closed-loop architecture which is designed to guarantee data persistence and sovereignty.

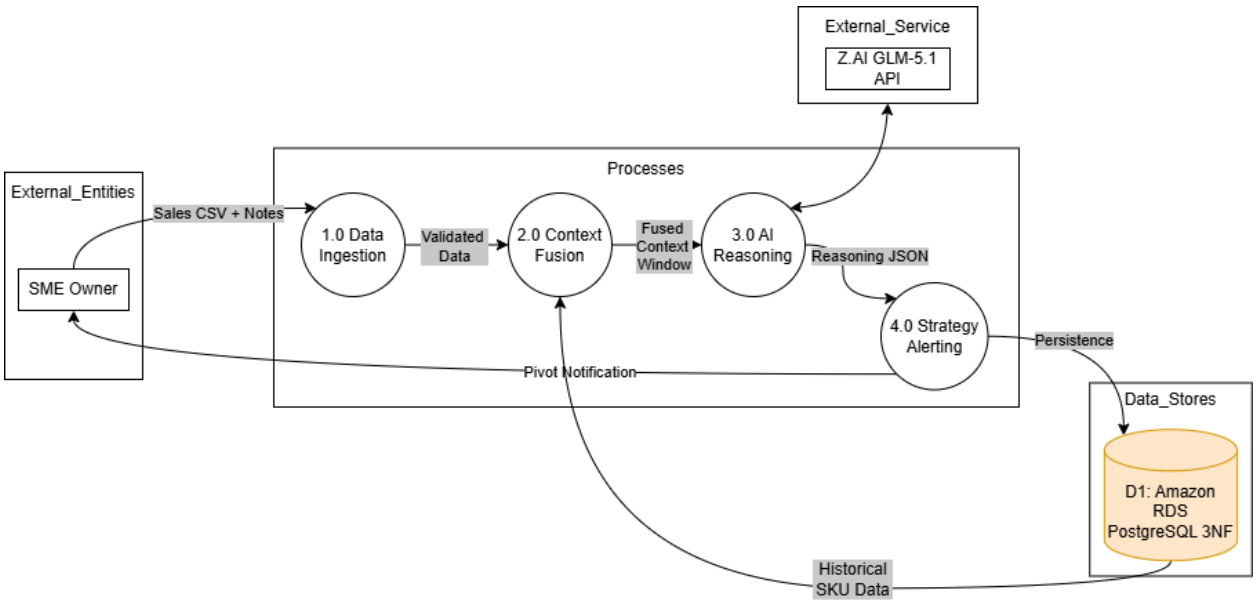

Figure 4: Data Flow Diagram

a. Interaction with External Entities & Services

- SME Owner (Entity): It is the main source of data. The process starts with the consumption of structured (Sales CSV) and unstructured (Contextual Notes) data. It is completed by issuing a Pivot Notification and the user is kept in a decision loop.
- ILMU GLM-5.1 API (External Service): It is the mandatory reasoning layer. The system is decoupled with this service and only exposes the “Fused Context” to it and gets back the reasoning JSON, which does not expose the underlying logic to the volatility of external APIs.

b. Process Decomposition & Data Transformation.

1. Process 1.0 (Data Ingestion): This process is the gatekeeper of this system. It does schema validation and sanitization. The output, Validated Data, makes sure that any downstream processes are not exposed to "Dirty Data" or malformed inputs.
2. Process 2.0 (Context Fusion): This is the logical centre of the system. It reads Historical SKU Data (D1) of the Amazon RDS and combines it with the real-time Validated Data.
3. Process 3.0 (AI Reasoning): This process deals with the two-way communication with the Z.AI GLM. It formats the prompt, controls the context window, and formats the raw model response into a standardized form of JSON.
4. Process 4.0 (Strategy Alerting): The last process deals with dual-output logic. It guarantees Persistence by re-writing the logic of the AI to the database to be audited again later and at the same time signifies the Pivot Notification to the frontend.

c. Compliance with Data Conservation Laws

To comply with the technical rubrics of "Proper Laws" the architecture employs the following protection measures:

- The Black Holes Prevention: All processes are tightly balanced. As an illustration, Process 2.0 takes two different data streams (Current and Historical) and combines them into one Fused Context Window. No data can be consumed without a state change or output.
- Prevention of "Miracles": No spontaneously generated data is created. Pivot Notification of Process 4.0 is functionally dependent on Reasoning JSON of Process 3.0 which is dependent on the context of Process 2.0.
- Data Store Integrity: Data can only flow in and out of the Amazon RDS (D1) and legitimate processes. The SME Owner does not have access to the database to manipulate the data in an unauthorized manner, which can ensure a strict Three-Tier Architecture.

2.3.2. Normalized Database Schema

The data layer is designed based on the Third normal form (3NF) to have absolute data integrity, no redundancy and high performance of queries. When these normalization standards are followed, the system will make sure that all the non-key attributes are functionally dependent only on the Primary Key.

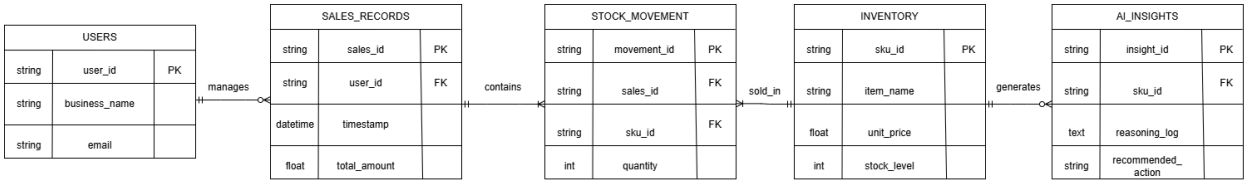

Figure 5: 3NF Entity Relationship Diagram

a. Multiple (M:N) Relationships Resolution.

One of the most important architectural features of the SmartStock AI is the application of the STOCK_MOVEMENT table. As a way of solving the complicated relationship between SALES_RECORDS and INVENTORY we added the STOCK_MOVEMENT junction table. This enables one sale to have many SKUs and one SKU to be part of many sales over time. This avoids the Black hole of data loss and will provide the correct tracking of stock of the Z.AI GLM to analyze.

b. Entity Definitions & Relationships

Entity	Role in Architecture	Relationship Logic
USERS	Identity Management	Acts as the parent entity; a single user manages multiple sales records.
SALES_RECORDS	Transaction Header	Captures the "who" and "when" of a sale. It is the bridge between the user and the stock movement.
STOCK_MOVEMENT	Transaction Detail	Records the "what" and "how much." It contains Foreign Keys (sales_id, sku_id) to ensure relational mapping.
INVENTORY	Resource Master	The "Golden Copy" of stock facts. Provides the structural data needed for Process 2.0 (Context Fusion) in the DFD.
AI_INSIGHTS	Reasoning Persistence	Stores the "Chain of Thought" reasoning logs. Linked to sku_id to allow for historical performance auditing of AI-generated strategies.

3. Functional Requirements & Scopes

3.1 Minimum Viable Product (MVP)

The SmartStock MVP is aimed at providing a high-impact AI-powered decision intelligence system within the hackathon duration. The chosen features portray the essential functionality and viability of the system.

Feature	Description
Data Input (CSV Upload / Manual Entry)	Structured sales and inventory information is uploaded by users, or data is manually keyed in together with contextual information (e.g., weather, events) to add further information.
Inventory Dashboard	Gives a centralized platform to see stock levels, past trends, and the performance of products in a simple, easy-to-use format.
AI Insights & Recommendations	The system uses Z.AI GLM to make demand forecasts and suggest optimal inventory choices based on both structured and unstructured data.
Explainable AI output	Every AI-generated suggestion is also explained in an understandable way, so the user can trust and comprehend the system's decisions.

3.2 Non-Functional Requirements (NFRs)

These define the quality attributes of the system to ensure reliability, performance, and scalability.

Quality	Requirement	Implementation
Scalability	The system should handle increasing data inputs and user requests efficiently.	Use a modular FastAPI backend with scalable deployment (e.g., containerization).
Reliability	The system should consistently generate valid and structured AI outputs.	JSON schema validation + retry mechanism for GLM responses.
Maintainability	Codebase should be modular and easy to update.	Separation of frontend, backend, and AI service layers.
Performance (Latency)	AI responses should be generated within 2–5 seconds.	Async API calls with timeout control (max 5s).
Cost Efficiency	Token usage per session must remain within budget.	Input summarization, prompt optimization, and caching repeated queries.
Usability	System must be easy for non-technical users.	Simple UI dashboard with minimal inputs and clear outputs.

3.3 Future Improvements / Out of Scope.

Although the SmartStock MVP is dedicated to the provision of the main AI-driven decision intelligence features within a restricted time span, some improvements have been outlined to be implemented in the future. The improvements are meant to enhance the automation of the systems, scalability, and real-world applicability as the product moves out of the prototype phase.

The introduction of real-time inventory monitoring with the help of IoT devices is one of the primary improvements. SmartStock can, in the future, be combined with physical sensors or smart inventory systems to update the stock levels in real time automatically. This would remove the manual entry of data, enhance the accuracy of data, and allow more dynamic and responsive decision-making.

The other valuable enhancement is the addition of multi-user functionality and role-based access control. This would enable various stakeholders in an organization, like managers, employees, and administrators, to interact with the system at the same time with varying degrees of access and privileges. This is a functionality that is needed to serve bigger SMEs and enhance teamwork within organizations.

The system can also be extended to support integration with enterprise resource planning (ERP) systems, such as SAP ERP and Oracle ERP. This would facilitate a smooth flow of data between SmartStock and business systems it has to work with, with automated data synchronization, better workflow, and less duplication of efforts.

Regarding analytics, next-generation versions can have more sophisticated hybrid forecasting engines, which integrate the logic of the Z.AI GLM using conventional machine learning methods like time-series forecasting. The hybrid would improve the accuracy of prediction, particularly with large-scale data, without compromising the explainability and contextual reasoning capabilities of the GLM.

Also, SmartStock can be expanded with automated external data integration, i. e., weather APIs and event databases. The system can obtain more accurate and proactive recommendations, automatically looking at real-time contextual data (e.g., weather conditions, public holidays, local events), without necessarily considering only the inputs provided by users.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

A systematic assessment and deployment plan is adopted to ensure that SmartStock can be relied upon and provide stable AI-based insights. The system is geared towards reducing risks during release and constant enhancement of performance through testing and monitoring.

4.1.1 Grayscale Rollout & A/B Testing

SmartStock uses a grayscale launch strategy which involves releasing new features to a small number of users initially, and then releasing them to all users. This will minimize the chances of a large-scale system failure and enable the team to assess the performance of the system within a controlled environment. Response time, system errors, and quality of AI-generated outputs are some of the key performance indicators that are closely monitored during this stage.

The rollout will involve:

- First release to a small group of users (~10%).

Continuous monitoring of:

- API response time
- Error rates
- Validity and applicability of AI decisions.
- Slow increase to a bigger number of users after stability is achieved.

Moreover, A/B testing is developed to enhance the effectiveness of the systems. The variants of prompt structures, as well as the recommendations format, are experimented on to find out which method yields more reliable and easy to use results.

4.1.2. Emergency Rollback & Golden Release.

In order to preserve system stability, SmartStock has an Emergency Rollback mechanism with a Golden Release strategy. When critical failures occur, the system is able to roll back to a previous stable version to have continuous service.

Under the following conditions, the roll-back mechanism is triggered:

- Poor success of AI responses.
- Invalid or unusable outputs of the GLM.
- System latency more than what is acceptable (e.g., over 5 seconds).

This is a strategy that will make sure that users still experience a reasonable experience despite the occurrence of unforeseen problems.

4.1.3 Priority Matrix

Priority	Condition	Action
P1 (Critical)	GLM API failure / no response	Trigger fallback response + rollback
P2 (High)	Incorrect or invalid AI output	Retry with refined prompt
P3 (Medium)	Slow response (>5s)	Show loading + optimize request
P4 (Low)	Minor UI issues	Fix in next update

4.2 Monitoring

SmartStock has in-built health monitoring and proactive detection of issues through ongoing monitoring of the system. This guarantees that the system behind the AI performs effectively and reliably.

The monitoring plan has a number of important elements:

1. **API Monitoring**

- Tracks to learn about how the system has been performing with respect to performance in terms of response time, volume of requests, and error rate in order to identify potential bottlenecks.

2. **AI Output Monitoring**

- Ensures that all work done by the GLM is in the required form and that it lacks anomalies and hallucinated answers.

3. **System Logging**

   Captures the important system activities, including:

- User inputs
- API interactions
- Error events

  These records aid in debugging and continuous improvement.

4. **Alert Mechanism**

   Automatically notifies developers when:

- The level of error is greater than pre-determined levels.
- GLM API is not available.

  The system has severe drops in performance.

4.3 Assumptions

The implementation and design of SmartStock are based on a number of assumptions that underlie the environment in which the system works.

These assumptions include:

- The system uses real-time API communication, which enables users to have stable internet connectivity.
- The appropriate and sufficiently filled data is input by the users, and it directly affects the quality of AI-generated insights.
- SMEs are receptive to the use of AI-assisted tools and are ready to implement recommendations in their decision-making process.
- The Z.AI GLM API is not closed and unreachable, being the main element that creates insights.

5. Project Management & Team Contributions

5.1 Project Timeline (10 Days)

Day	Task
Day 1–2	Planning, idea validation, PRD & SAD drafting
Day 3–4	Backend setup (FastAPI + database schema)
Day 5–6	Frontend development (dashboard + input forms)
Day 7–8	GLM integration & prompt design
Day 9	Testing, debugging, optimization
Day 10	Final polishing, documentation, and presentation

5.2 Team Members' Role

Member Name	Role
Maha Vaarshinie A/P Arupatha Rajoo	Frontend & UI/UX Developer
Nareenthiran A/L Bhuvaneswaran	Backend & Data Engineer
Yasha Durgashine	AI Architect

5.3 Recommendations

In order to optimize the performance of the system, its scalability, and durability, a number of suggestions should be made when developing the system in the future.

To start with, the use of a caching system, like Redis, can drastically enhance the performance because it will decrease the frequency of API calls and minimise the costs of tokens. It can be applied especially in processing repeated queries or other user inputs.

Moreover, external data sources (weather APIs and local event data) would enhance the contextual awareness of the system and allow for more accurate predictions of the demand. This would decrease dependence on context provided by the user and increase automation.

This can be enhanced further by adding hybrid methods of forecasting, whereby the reasoning abilities of the GLM are integrated with the traditional machine learning models. This would enhance accuracy in predictions and system robustness, particularly with huge datasets.

Lastly, the SmartStock implementation on the cloud infrastructure platform, like AWS or Google Cloud Platform (GCP), would be more scalable and reliable. The features that Cloud deployment will support with auto-scaling, load balancing, and high availability are necessary to support more users and larger data volumes.